1 Orbit and Clock Product Evaluation

Every processing mode in the engine (post-processed PPP, PPP-RTK, the baseline modes, and the real-time engine) needs a position $\boldsymbol{r}^i$, velocity $\dot{\boldsymbol{r}}^i$, and clock bias δt^i for each satellite at each query epoch. These are evaluated *before* any state-space refinement. In post-processing they are final. In the real-time engine they are the broadcast values that the SSR overlay (the SSR corrections document) corrects with a RAC orbit delta and a scalar clock delta. This document derives the two precise interpolation paths, the two broadcast evaluation paths, and the ephemeris selection that feeds them. It stops where the SSR delta is added: the RAC frame, the IOD gate, and the freshness checks are in the SSR document. The Sagnac (Earth-rotation) range correction, which makes $\boldsymbol{r}^i$ at transmit time consistent with the line of sight, is in the observation-physics document. The ERP and polar-motion transforms between the terrestrial and celestial frames are in the reference-frames document.

There is a *precise* path and a *broadcast* path. The precise path *interpolates* tabulated SP3 orbit samples and CLK clock samples (IGS Final / Rapid products). It is the post-processing default and is accurate to centimetres or better. The broadcast path evaluates a Keplerian ephemeris from the navigation message, or for GLONASS a numerically integrated PZ-90 ephemeris. It is accurate to metres and is what the live SSR stream corrects. By default the two paths do not mix for one satellite at one epoch: a loaded precise product that spans the query is used, and otherwise the engine falls back to the broadcast evaluation (`src/core/model/ModelGeometry.cpp`, `src/core/corrections/GnssCorrections.cpp`). The choice is made per quantity, though. The real-time RTB-4 gate `realtime.use_precise_orbits` selects the orbit and clock sources independently (`GnssConfig::realtimeUsesLiveOrbitSource` vs `realtimeUsesLiveOrbitClockSource`, `ProductParser::loadProducts(bool, bool)`). With that gate armed, an "ultra-rapid orbits + network clocks" configuration uses a precise interpolated orbit *and* a broadcast+SSR clock for the same satellite and epoch. The flag defaults to false. The orbit gate then returns the same value as the clock gate, the result is byte-identical, and the no-mixing statement holds for every non-real-time mode and for the default real-time path.

1.1 Precise orbit: derotated Neville interpolation

SP3 files tabulate the satellite position in the ECEF frame on a coarse grid (typically 900 s for IGS Final, 300 s for Rapid). The ECEF trajectory is *not* a smooth low-degree polynomial. The rotating frame adds an oscillation at the sidereal frequency, and a degree-10 fit cannot follow it to better than a few millimetres near the SP3 epoch boundaries. The engine therefore *derotates* each node into a common near-inertial frame anchored at the query epoch before interpolating (`src/core/model/ModelGeometry.cpp`, following the RTKLIB / cssrlib convention [4]). For a node sampled at t_i and a query at t_q, with the Earth-rotation rate $\omega_\oplus = 7.2921151467 \times 10^{-5}$ rad/s (`Constants::EARTH_ANGULAR_VELOCITY`), the in-plane coordinates are rotated by the angle the frame swept between the two epochs:

$$\boldsymbol{r}^i_{\mathrm{I}}(t_i) = \boldsymbol{R}_z(\omega_\oplus\,(t_i - t_q))\;\boldsymbol{r}^i(t_i), \qquad \boldsymbol{R}_z(\alpha) = \begin{bmatrix} \cos\alpha & -\sin\alpha & 0 \\ \sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{1}$$

Concretely, with $\alpha_i = \omega_\oplus\,(t_i - t_q)$ the derotated in-plane components written to the interpolation arrays are $\tilde{x}_i = \cos\alpha_i\, x_i - \sin\alpha_i\, y_i$ and $\tilde{y}_i = \sin\alpha_i\, x_i + \cos\alpha_i\, y_i$, with z unchanged (`ModelGeometry.`

cpp). The rotation angle is zero at $t_i = t_q$, so the interpolated value at the query is already the ECEF position at t_q and needs no rotation back.

Bracket-centred window. The choice of node window determines whether the interpolated position is *continuous*. The engine selects an IGS_POINTS = 11-node window centred on the *bracket* $[t_a, t_{a+1}]$ that contains t_q, not the 11 nodes nearest to t_q (`ModelGeometry.cpp`). A nearest-N rule changes a node at the *midpoint* between SP3 epochs. There the two candidate polynomials disagree by a few millimetres, which produces the known 15-minute sawtooth in PPP position, synchronous across constellations. A bracket-centred rule changes the window when t_q crosses a node t_{a+1}. That node belongs to both windows, so the Neville value is the same on either side and the orbit is continuous. A gap guard re-picks a narrower MIN_POINTS = 9-node window and rejects the satellite if the residual span still exceeds the fit interval (thresholds in `ModelGeometry.cpp`; tuning values cited, not reproduced). The "fit interval" is not read from the product header. It is the *median* of the leading consecutive-node spacings (`ModelGeometry.cpp`). The reason is that a multi-file SP3 merge can be non-uniformly sampled. For example, a 5-minute ultra-rapid file concatenated with a 15-minute one keeps the finer file's off-grid nodes, because the 1 s dedup window correctly treats them as distinct (`ProductParser` warns on the mixed-rate concat). The median spacing gives a stable MAX_SPAN guard on such a series, where a nominal grid is not defined.

Neville value and derivative. Within the window the engine runs Neville's algorithm simultaneously for the value and its analytic derivative (`src/core/math/Interpolate.cpp`, `Interpolate::nevilleInterpDeriv`). The node abscissae are shifted to $s_i = t_i - t_q$ to avoid cancellation: raw continuous GPS seconds are $\sim 1.36 \times 10^8$, so an unshifted difference loses ~ 8 digits. The shift changes neither the polynomial value nor its derivative. With the tableau column P_i for the value and D_i for the derivative, the coupled recurrence over degree j (evaluating at the shifted query $s_q = 0$) is

$$P_{i,j} = \frac{(s_q - s_{i+j})\, P_{i,j-1} - (s_q - s_i)\, P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{2}$$

$$D_{i,j} = \frac{(s_q - s_{i+j})\, D_{i,j-1} + P_{i,j-1} - (s_q - s_i)\, D_{i+1,j-1} - P_{i+1,j-1}}{s_i - s_{i+j}}, \tag{3}$$

seeded by $P_{i,0} = y_i$ and $D_{i,0} = 0$; the converged scalars are $P_{0,n-1}$ and $D_{0,n-1}$ (`Interpolate.cpp`). The derivative recurrence (3) is $\partial/\partial s_q$ of the value recurrence (2), so one pass gives both and no finite difference is needed.

Inertial-to-ECEF velocity. The Neville derivative is taken on the *derotated* channels, so it is the velocity in the inertial frame fixed at t_q. The Doppler line-of-sight rate, the orbit-normal attitude, and the relativistic $\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i$ term need the ECEF velocity, which follows by removing the frame rotation $\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \dot{\boldsymbol{r}}^i_{\mathrm{I}} - \boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i$. For $\boldsymbol{\omega}_\oplus = (0, 0, \omega_\oplus)$ this is the boxed transform applied in `ModelGeometry.cpp`:

$$\boxed{\dot{\boldsymbol{r}}^i_{\mathrm{ECEF}} = \begin{bmatrix} v_{\mathrm{I},x} + \omega_\oplus\, \boldsymbol{r}^i_y \\ v_{\mathrm{I},y} - \omega_\oplus\, \boldsymbol{r}^i_x \\ v_{\mathrm{I},z} \end{bmatrix}}. \tag{4}$$

The relativistic clock term is unaffected, since $\boldsymbol{r}^i \cdot (\boldsymbol{\omega}_\oplus \times \boldsymbol{r}^i) = 0$. Only the Doppler and attitude computations see the difference (`ModelGeometry.cpp`).

1.2 Precise orbit: sidereal-trigonometric LSQ fit

A config-gated alternative (`config.enableTrigSp3Interp()`) replaces the Neville polynomial with an over-determined least-squares fit on a *sidereal-trigonometric basis*, run on the identical derotated channels and time base (`ModelGeometry.cpp`; `Interpolate::trigSvdInterp`). The basis is a constant plus four sine/cosine harmonic pairs of the fundamental sidereal angular rate, nine terms in all:

$$\boldsymbol{r}_{\mathrm{c}}^{i}(\tau) = c_0 + \sum_{f=1}^{4}\big[c_{2f-1}\sin(f\,\Omega_0\,\tau) + c_{2f}\cos(f\,\Omega_0\,\tau)\big], \qquad \tau = t - t_0, \tag{5}$$

where t_0 is the first node. The fitted curve does not depend on this choice, because the constant term absorbs the offset and the sine and cosine pairs absorb the phase. The fundamental rate is the angular frequency of the *mean sidereal day*. With the sidereal-day ratio $\kappa = 0.99726956634$ and a nominal solar day of 86400 s,

$$\boxed{\Omega_0 = \frac{2\pi}{\kappa \cdot 86400\ \mathrm{s}} \quad [\mathrm{rad/s}], \qquad \kappa = 0.99726956634,} \tag{6}$$

expressed *per second*, so τ stays in the caller's seconds and the analytic derivative is per second (`Interpolate.cpp`). M-PAGES uses rad/day and divides the velocity by 86400, which is numerically identical. The constant $\kappa = 0.99726956634$ is the IERS ratio of the mean sidereal day to the solar day. The model depends on it directly, so it is quoted here.

The fit is solved by Eigen's `JacobiSVD` with a relative singular-value threshold of 10^{-15}, which yields the minimum-norm least-squares solution, the Numerical-Recipes `svbksb` convention M-PAGES' `svdfit` uses [3]. Per-node weights derive from the SP3 `++` accuracy codes; because those are a single value per satellite, the weight set is uniform and the fit is effectively unweighted, matching M-PAGES. The analytic value and derivative at the query $\tau_q = t_q - t_0$ are

$$\boldsymbol{r}_{\mathrm{c}}^{i}(\tau_q) = c_0 + \sum_{f=1}^{4}\big[c_{2f-1}\sin(f\Omega_0\tau_q) + c_{2f}\cos(f\Omega_0\tau_q)\big], \tag{7}$$

$$\dot{\boldsymbol{r}}^{i}{}_{\mathrm{c}}(\tau_q) = \sum_{f=1}^{4}(f\Omega_0)\big[c_{2f-1}\cos(f\Omega_0\tau_q) - c_{2f}\sin(f\Omega_0\tau_q)\big], \tag{8}$$

with the per-harmonic angular weight $w_f = f\,\Omega_0$ (`Interpolate.cpp`). The fit is an *over-determined smoother*. The window has up to 11 nodes against 9 basis terms, so for $N > 9$ the fitted curve does *not* pass through the nodes. This differs from the M-PAGES 9-point exact fit, and is kept so that the trig branch can reuse the derotated window unchanged. The same inertial-to-ECEF velocity conversion (4) is then applied to $\dot{\boldsymbol{r}}^{i}{}_{\mathrm{c}}$.

1.3 Precise clock interpolation

The CLK product tabulates the scalar satellite clock bias δt^i on its own grid (commonly 30 s). Clock noise behaves like a random walk in bias with occasional rate discontinuities, so the engine offers three interpolation modes, selected by `config.getSatelliteClockInterpolationMode()` (`src/core/corrections/GnssCorrections.cpp`). All three are bracket-centred for the same reason as the orbit window: the window must change at a shared node, never at a midpoint between epochs.

(a) Linear (MMSE for Brownian motion). The default is a two-node linear interpolation across the bracket $[t_0, t_1]$ containing t_q:

$$\delta t^i(t_q) = \delta t_0^i + \left(\delta t_1^i - \delta t_0^i\right) \frac{t_q - t_0}{t_1 - t_0}, \tag{9}$$

which is the minimum-mean-square-error estimator for a Wiener process sampled at the two bracketing epochs. If the gap exceeds a guard threshold the interpolation is rejected (`GnssCorrections.cpp`; gap value cited, not reproduced).

(b) Rate-anchored weighted LSQ. The `LsqSatClock` mode fits a local line $\delta t^i(t) \approx a + b\,(t - t_q)$ over an $\sim$11-node window by weighted least squares, with the centring shift $s_k = t_k - t_q$ making the fitted value at the query equal to the intercept a. The CLK product may also carry a clock *rate* m_j at the coarse 5-minute epochs. Each rate is added as a direct observation of the slope, $m_j = b + \mu_j$. A value equation $y_k = a + b\,s_k + \nu_k$ has $\partial/\partial a = 1$ and $\partial/\partial b = s_k$. A rate equation has $\partial/\partial a = 0$ and $\partial/\partial b = 1$, so the rate observations enter the slope row only. With value weights $w_k^{(y)} = 1/\sigma_{y,k}^2$ and rate weights $w_j^{(m)} = 1/\sigma_{m,j}^2$, the 2×2 weighted normal equations are

$$\boxed{\begin{bmatrix} \sum_k w_k^{(y)} & \sum_k w_k^{(y)} s_k \\ \sum_k w_k^{(y)} s_k & \sum_k w_k^{(y)} s_k^2 \;+\; \sum_j w_j^{(m)} \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} \sum_k w_k^{(y)} y_k \\ \sum_k w_k^{(y)} s_k y_k \;+\; \sum_j w_j^{(m)} m_j \end{bmatrix}}, \tag{10}$$

solved in closed form for the intercept

$$a = \frac{\left(S_{wxx} + S_m\right) S_{wy} - S_{wx}\left(S_{wxy} + S_{my}\right)}{S_w\left(S_{wxx} + S_m\right) - S_{wx}^2}, \tag{11}$$

with the running sums named exactly as in `GnssCorrections.cpp` ($S_w = \sum w_k^{(y)}$, $S_{wx} = \sum w_k^{(y)} s_k$, $S_{wxx} = \sum w_k^{(y)} s_k^2$, $S_{wy} = \sum w_k^{(y)} y_k$, $S_{wxy} = \sum w_k^{(y)} s_k y_k$, $S_m = \sum w_j^{(m)}$, $S_{my} = \sum w_j^{(m)} m_j$). If the determinant in the denominator vanishes (a window of coincident times) the code falls back to linear interpolation. The estimate $\delta t^i(t_q) = a$ is returned. The fit averages $\sim N$ samples where linear interpolation uses two, so the observation-noise part of the estimate at the bracket midpoint falls by $\sim\sqrt{N/2}$ relative to (9). This matters over seconds to minutes, where CLK observation noise exceeds the per-sample process noise. The cost is that the fit no longer passes through the nodes, so millimetre-level discontinuities appear at the clock epochs when the window moves.

(c) Barycentric Lagrange. For high-rate (1 Hz) products a barycentric-form Lagrange interpolation over a LAGRANGE_POINTS = 9-node window is available (`GnssCorrections.cpp`; `Interpolate::barycentricWeights` / `barycentricEval`). The barycentric form

$$\delta t^i(t_q) = \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}\,\delta t_i^i \;\Bigg/\; \sum_{i=1}^{n} \frac{w_i}{t_q - t_i}, \qquad w_i = \prod_{k \neq i} \frac{1}{t_i - t_k}, \tag{12}$$

is the numerically stable evaluation of the degree-$(n-1)$ Lagrange polynomial [1]. An extrapolation guard falls back to two-node linear interpolation when t_q lies outside the node span. This mode is *not* recommended for standard 30 s IGS Final clocks, because the polynomial oscillates between sparse samples. It is kept for high-rate input only.

1.4 Broadcast Keplerian evaluation

When no precise product spans the query, GPS, Galileo, BeiDou and QZSS are evaluated from their Keplerian broadcast ephemeris (`src/core/products/LiveEphemerisStore.cpp`, `keplerPosition`). The live SSR stream corrects this evaluation. Each system carries its own gravitational parameter μ and Earth-rotation rate $\omega_\oplus$ (`LiveEphemerisStore.cpp`): GPS / QZSS use $\mu = 3.986005 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, $\omega_\oplus = 7.2921151467 \times 10^{-5}\,\mathrm{rad/s}$; Galileo / BeiDou use $\mu = 3.986004418 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, with BeiDou taking the slightly different $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$. BeiDou is evaluated on its native BeiDou Time (BDT) scale, where BDT week 0 is GPS week 1356 and BDT = GPS − 14 s. Both the query epoch and the stored t_{oe} stay on BDT, so the node-rotation term below uses BDT seconds-of-week (`LiveEphemerisStore.cpp`).

With the semi-major axis $a = (\sqrt{a})^2$ and the time-from-ephemeris $t_k = t - t_{oe}$ reduced into $[-302400, +302400]\,\mathrm{s}$ (week rollover), the corrected mean motion adds the broadcast δn to the Keplerian value, and the mean anomaly propagates linearly:

$$n = \sqrt{\frac{\mu}{a^3}} + \delta n, \qquad M = M_0 + n\,t_k. \tag{13}$$

Kepler's equation $M = E - e\sin E$ is solved by Newton iteration (up to 15 steps, convergence at $|\Delta E| < 10^{-13}$):

$$E_{k+1} = E_k + \frac{M - \big(E_k - e\sin E_k\big)}{1 - e\cos E_k}. \tag{14}$$

The true anomaly and the argument of latitude follow, then the broadcast second-harmonic corrections refine the argument of latitude u, the orbital radius r, and the inclination i:

$$\nu = \operatorname{atan2}\big(\sqrt{1-e^2}\,\sin E,\ \cos E - e\big), \qquad \phi = \nu + \omega, \tag{15}$$

$$\begin{aligned} u &= \phi + c_{us}\sin 2\phi + c_{uc}\cos 2\phi, \\ r &= a\big(1 - e\cos E\big) + c_{rs}\sin 2\phi + c_{rc}\cos 2\phi, \\ i &= i_0 + \dot{i}\,t_k + c_{is}\sin 2\phi + c_{ic}\cos 2\phi. \end{aligned} \tag{16}$$

The corrected longitude of the ascending node includes the node rate *relative to* the rotating Earth and the fixed node offset at the ephemeris epoch:

$$\boxed{\Omega = \Omega_0 + \big(\dot{\Omega} - \omega_\oplus\big)\,t_k - \omega_\oplus\,t_{oe},} \tag{17}$$

where the final $-\omega_\oplus\,t_{oe}$ uses the seconds-of-week of t_{oe} on the native scale (`LiveEphemerisStore.cpp`). With the in-plane position $(x_o, y_o) = (r\cos u,\ r\sin u)$ the ECEF position is the standard rotation through Ω and i:

$$\boldsymbol{r}^i = \begin{bmatrix} x_o\cos\Omega - y_o\cos i\,\sin\Omega \\ x_o\sin\Omega + y_o\cos i\,\cos\Omega \\ y_o\sin i \end{bmatrix}. \tag{18}$$

The additional fixed rotation for BeiDou GEO satellites is not applied, because GEO satellites are rarely used for PPP-RTK. Velocity is a central finite difference of (18), $\dot{\boldsymbol{r}}^i \approx [\boldsymbol{r}^i(t+\tfrac{1}{2}) - \boldsymbol{r}^i(t-\tfrac{1}{2})]/(2\,\Delta t)$ with $\Delta t = 0.5\,\mathrm{s}$, using the same position evaluator (`LiveEphemerisStore.cpp`).

1.5 Broadcast GLONASS: PZ-90 RK4 integration

GLONASS does not broadcast Keplerian elements. It broadcasts an ECEF state vector (position, velocity, and a luni-solar acceleration) valid at t_b, and the ICD prescribes numerical integration of the PZ-90 equations of motion to the query epoch [2]. The engine uses a fixed-step fourth-order Runge–Kutta integrator (`src/core/products/LiveEphemerisStore.cpp`, `glonassState`). The per-axis acceleration combines the central gravitational term, the J_2 oblateness perturbation, the centrifugal and Coriolis terms of the rotating frame, and the broadcast luni-solar acceleration held constant over the integration. Writing $r = \|\boldsymbol{r}^i\|$ and the oblateness coefficient

$$\kappa_{J_2} = \tfrac{3}{2}\, J_2\, \frac{\mu\, a_e^2}{r^5}, \qquad J_2 = 1.0826257 \times 10^{-3}, \tag{19}$$

(with $\mu = 3.9860044 \times 10^{14}\,\mathrm{m}^3/\mathrm{s}^2$, $a_e = 6378136\,\mathrm{m}$, $\omega_\oplus = 7.292115 \times 10^{-5}\,\mathrm{rad/s}$, all PZ-90 constants at `LiveEphemerisStore.cpp`), the accelerations are

$$\boxed{\begin{aligned} \ddot{x} &= -\frac{\mu}{r^3}x + \kappa_{J_2}\, x\big(1 - 5\,z^2/r^2\big) + \omega_\oplus^2 x + 2\omega_\oplus \dot{y} + a_x, \\ \ddot{y} &= -\frac{\mu}{r^3}y + \kappa_{J_2}\, y\big(1 - 5\,z^2/r^2\big) + \omega_\oplus^2 y - 2\omega_\oplus \dot{x} + a_y, \\ \ddot{z} &= -\frac{\mu}{r^3}z + \kappa_{J_2}\, z\big(3 - 5\,z^2/r^2\big) + a_z, \end{aligned}} \tag{20}$$

where (a_x, a_y, a_z) is the broadcast luni-solar acceleration. The oblateness term on the z axis has the factor $3 - 5z^2/r^2$, against $1 - 5z^2/r^2$ on the in-plane axes. The centrifugal $\omega_\oplus^2$ and Coriolis $2\omega_\oplus$ terms appear only on the in-plane axes, as the PZ-90 rotating frame requires. The integrator uses a maximum step of $h = 30\,\mathrm{s}$, signed toward the query, and shortens the last step so that it ends on t_q. Each step is the classical RK4 update $y \leftarrow y + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ (`LiveEphemerisStore.cpp`).

1.6 Broadcast satellite clock

The broadcast satellite clock for the Keplerian systems is the standard second-order polynomial in the time-from-clock-epoch $t - t_{oc}$ (on the native scale, with BeiDou on BDT as for the orbit node term):

$$\boxed{\delta t^i = a_{f0} + a_{f1}\,(t - t_{oc}) + a_{f2}\,(t - t_{oc})^2.} \tag{21}$$

GLONASS instead uses its two-term broadcast model with the 1020-message τ_n (DF123) and γ_n (DF120) about $t_b = t_{oe}$:

$$\delta t^i = -\tau_n + \gamma_n\,(t - t_b) \tag{22}$$

(`LiveEphemerisStore.cpp`). The shipped real-time replay configurations exclude GLONASS from `active_systems`, so they do not use this path. The default `active_systems` and the post-processing configurations enable GLONASS and do use it.

The precise-clock-baseline convention. The broadcast clock (21) *omits* both the periodic relativistic eccentricity term and the group delay (TGD/BGD). This is intentional. The IGS-RTS clock correction is generated against the bare polynomial, so the engine must *not* apply the periodic relativity or TGD before adding the SSR delta. Doing so would count them twice. The periodic relativistic term $-2\,\boldsymbol{r}^i \cdot \dot{\boldsymbol{r}}^i / c^2$ is added later, in the observation-physics path; see the observation-physics document for its derivation. With this convention the orbit, the bare clock, and the SSR overlay all refer to the same ephemeris.

1.7 Ephemeris selection and IOD bookkeeping

The broadcast paths assume that the correct ephemeris record has been selected. The live store holds a `current` and an optional `previous` record per satellite. The rules below decide which one is evaluated (`src/core/products/LiveEphemerisStore.cpp`).

Freshest-toe within a source. Within one source, the record with the newer reference epoch is kept. An incoming record with an older t_{oe} (a stale re-broadcast) is rejected. A record with the same epoch, or one differing by less than a second, overwrites the held record (`LiveEphemerisStore.cpp`). At query time the store chooses between `current` and `previous` by *closest* t_{oe} on a common GPS-continuous scale. BeiDou's BDT t_{oe} is first converted to GPS seconds (`toeGpsContinuous`). Without that, every BeiDou record would appear $\sim$1356 weeks old and be skipped.

Fit window. A held record is evaluated only within a per-system limit on $|t - t_{\mathrm{oe}}|$, about twice the broadcast fit interval of the system and longest for GPS and QZSS, shortest for GLONASS. Beyond it the satellite is skipped and the caller keeps its prior value, so that a stale record never produces a wrong position (`LiveEphemerisStore::withinFitWindow`).

Selection by transmission time. With `network.apriori_source = sp3_brdc_clock` (SP3 orbits, the broadcast clock, no CLK product; default `sp3`, not set by the reference configurations) the record of satellite s at epoch t is the latest healthy one whose transmission time is at or before t (`network.brdc_select = ttom`). The a priori clock is kept continuous across every cut-over by an offset table,

$$a_s(t) = \mathrm{brdc}_k(t) + O_{s,k}, \qquad O_{s,k} = O_{s,k-1} + \mathrm{brdc}_{k-1}(t_{\mathrm{sw}}) - \mathrm{brdc}_k(t_{\mathrm{sw}}), \tag{23}$$

with t_{sw} the first 30 s node at or after the transmission time of record k and brdc_k the polynomial (21), so the estimator sees no step. The emitted clock refers to the stamped record, $C_{\mathrm{prod}} = C_{\mathrm{est}} + O_{s,k}$ with `iode` $= k$, and a user holding record k reconstructs $\mathrm{brdc}_k(t) + C_{\mathrm{prod}} = a_s(t) + C_{\mathrm{est}}$ (`products/BrdcClockApriori.hpp`). The store is fed in transmission order (`LiveEphemerisStore::insertTransmitted`): the record replaces the current one whatever its t_{oe}, because the freshest-t_{oe} rule above rejects a re-upload whose t_{oe} is earlier than that of the held record.

IGS-source authority over receiver nav. When the engine ingests both an IGS broadcast stream and the receiver's own decoded navigation, the IGS record takes priority. The IGS-RTS SSR delta is computed against the IGS broadcast ephemeris, so the rover must pair the delta with the *same* ephemeris. Even at an identical IOD, the receiver's locally assembled orbit has slightly different fit parameters: a live Galileo I/NAV assembly was observed to differ by more than 100 m. Applying IGS-SSR to a receiver ephemeris therefore corrupts the single-difference observations. A receiver record never overwrites a current IGS record with an equal or older t_{oe}. It is accepted only when it is newer than the IGS record by more than one constellation refresh (1800 s, the GLONASS nominal step), which indicates a *stalled* IGS feed. A dead IGS stream then falls back to receiver navigation and the satellite stays usable (`LiveEphemerisStore.cpp`).

IODE-change demote and SSR re-referencing. When the IOD changes, the `current` record becomes `previous` and is not discarded, so the superseded orbit remains available across the cutover (`LiveEphemerisStore.cpp`). This is needed because the receiver can hold an ephemeris one version ahead of the SSR stream. When an SSR delta names an IOD, `pickForSsr` returns whichever held

record (`current` or `previous`) matches it. The orbit and clock are then evaluated on the ephemeris the correction was computed for, and the satellite is not dropped to broadcast accuracy because `current` and the SSR disagree. Keeping both records also supports a second use of the broadcast clock polynomial (21): `LiveEphemerisStore::ephCutoverShiftMetres` evaluates $c\,(\delta t^{i}_{\text{new}} - \delta t^{i}_{\text{old}})$ across an ephemeris version change (`computeSatelliteClock` run once on the freshest pick and once pinned to the old IODE), and returns nothing when the old version is no longer held. The network producer's `network.apriori_source = brdc_rts` eph-cutover re-level uses this value. That re-level is derived in the Network mode document.

The GNSS-IOD matching convention. The SSR "GNSS IOD" field (IGS-SSR / RTCM `4076`, IDF012) is an 8-bit quantity whose definition differs by constellation and differs from the raw broadcast IODE stored with the ephemeris. `ssrMatchIode` converts the held record's IODE to the SSR convention, in one place (`LiveEphemerisStore.cpp`):

$$\mathrm{IOD}_{\mathrm{SSR}}(i) = \begin{cases} \text{IODE} & \text{GPS, QZSS (8-bit LNAV, passthrough),} \\ \text{IODnav \& } \texttt{0xFF} & \text{Galileo (low 8 bits of the 10-bit I/NAV IODnav),} \\ \left\lfloor t_{oe}^{\mathrm{sow}}/720 \right\rfloor \bmod 240 & \text{BeiDou (toe-derived, per IGS-SSR v1.00),} \end{cases} \tag{24}$$

with t_{oe}^{sow} the BDT seconds-of-week. The BeiDou broadcast AODE that the decoder stores in the IODE field is a *different* quantity and never matches the SSR field, so the toe-derived value is used instead. The raw IODE is kept for the record bookkeeping above; only the value passed to the SSR gate is converted. The SSR IOD-gate section describes how that value gates the SSR orbit and clock delta: the delta is accepted only on an exact IOD match, and otherwise the satellite falls back to broadcast accuracy.

References

[1] Jean-Paul Berrut and Lloyd N. Trefethen. "Barycentric Lagrange Interpolation". In: *SIAM Review* 46.3 (2004), pp. 501–517. DOI: 10.1137/S0036144502417715.

[2] *GLONASS Interface Control Document: Navigational Radiosignal in Bands L1, L2 (Edition 5.1)*. Russian Institute of Space Device Engineering. Moscow, 2008.

[3] William H. Press et al. *Numerical Recipes: The Art of Scientific Computing*. 3rd. Cambridge: Cambridge University Press, 2007.

[4] Tomoji Takasu and Akio Yasuda. "Development of the Low-Cost RTK-GPS Receiver with an Open Source Program Package RTKLIB". In: *International Symposium on GPS/GNSS*. Jeju, Korea, 2009, pp. 4–6.